

DVC - lakeFS Karşılaştırması

Bu çalışma taslağının ilk bölümü, araç karşılaştırmasına geçmeden önce veri versiyonlama ihtiyacını doğuran proje kısıtlarını tanımlar.

1. Constraints / Gereksinim Kısıtları

C1

Veri statik değildir

Ham veri sisteme batch'ler halinde sürekli akar. Bu nedenle processed dataset tek sefer üretilen sabit bir çıktı olarak değerlendirilemez; yeni ham veri geldikçe yeni bir yayınlanabilir data snapshot'ı oluşabilir.

C2

Processed data iki farklı nedenle değişebilir

Yeni bir raw batch aynı pipeline ile işlendiğinde veya mevcut raw snapshot yeni bir pipeline sürümüyle yeniden işlendiğinde processed data değişebilir. Versiyon geçmişi, bu iki değişim nedenini birbirinden ayırt edebilmelidir.

Değişim kaynağı	Girdi	Beklenen sonuç
Yeni ham veri	Yeni batch + aynı pipeline	Yeni processed data sürümü
Pipeline kodu değişikliği	Aynı raw snapshot + yeni pipeline	Yeni processed data sürümü

C3

Data ve pipeline sürümleri bağımsız kimliklerdir

Data version, yayınlanan processed snapshot'ı; pipeline version ise bu snapshot'ı üreten processing kodunun release'ini temsil eder. Tek bir birleşik sürüm numarası yerine iki kimlik ayrı tutulmalı ve lineage metadata'sı üzerinden bağlanmalıdır.

C4

Her data tag'i üretici kodu izlenebilir kılmalıdır

Processed data tag metadata'sı en az data sürümünü, pipeline sürümünü, pipeline Git commit SHA'sını ve kullanılan raw batch/snapshot bilgisini taşımalıdır. Böylece bir data sürümündeki değişimin kaynağı yeniden üretilebilir biçimde belirlenebilir.

Önerilen minimum lineage kaydı

```
tag: processed-data-v1.4.0
metadata:
  pipeline_version: pipeline-v1.3.0
  pipeline_git_commit: abc123
  raw_batches: 1-101
```

2. Use Cases ve Karşılaştırma

U1

Tek geliştirici, tek canonical dataset

History lineer ilerler. Her yayınlamada tek bir .dvc pointer'ı Git commit'i içinde güncellenir; önceki data sürümü ilgili commit/tag'den pointer restore edilip dvc checkout ile alınabilir. Bu modelde DVC doğrudan ve yeterli bir çözümdür. Aynı bir data versioning servisi kurmak lakeFS'i operasyonel olarak overkill hale getirebilir.

U2

Birden fazla geliştirici, alternatif processing çalışmaları

İzolasyon yalnızca lakeFS'e özgü değildir. DVC'de her geliştirici ayrı Git branch'inde kendi .dvc pointer'ını ve pipeline kodunu ilerletebilir; seçilen sonuç main'e merge edilir. Yani teknik olarak aynı geliştirme akışı DVC ile de kurulabilir. lakeFS'in farkı, data branch'lerini Git branch'lerinden bağımsız, merkezi ve zero-copy olarak sunması; aynı branch/commit'i çok sayıda consumer'ın object storage arayüzünden doğrudan okuyabilmesidir.

TEME
AYRIM

DVC ile data branch davranışı Git branch'leri ve pointer'lar üzerinden modellenilebilir. lakeFS'in belirgin değeri, bu branch modelini object storage katmanında zero-copy ve çoklu consumer erişimine hazır bir servis olarak sağlamasıdır.

3. DVC | lakeFS Karşılaştırması

Kriter	DVC	lakeFS
Kod-data ilişkisi	+ .dvc pointer'ı Git commit'ine gömülür; kod ve data referansı doğal olarak aynı history'de buluşur.	+ Kod ve data lifecycle'ları bağımsızdır. - lakeFS commit/tag'i ile pipeline Git SHA'sı metadata üzerinden ayrıca eşleştirilmelidir.
Kurulum ve işletim	+ Git repo + DVC remote ile başlar; ayrı bir versioning server'ı işletmek gerekmez. Kurulum ve zihinsel model daha basittir.	- Paylaşılan production kullanımında erişilebilir bir lakeFS servisi gerekir: self-hosted deployment veya managed cloud. Self-hosted model metadata DB, izleme ve yedekleme sorumluluğu getirir.
Branch ve paralellik	+ Git branch'leri ve ayrı pointer'larla izolasyon teknik olarak sağlanır. - Data branch yaşam döngüsü Git workflow'una bağlı kalır.	+ Branch oluşturmak metadata seviyesinde zero-copy'dir; değişmeyen objeler kopyalanmaz, branch'ler merkezi ve bağımsız data ortamlarıdır.
Çoklu consumer	- Consumer'ın doğru Git/DVC ref'ini çözmesi ve veriyi remote/cache üzerinden çalışma ortamına alması veya DVC entegrasyonu kullanması gerekir.	+ Birden fazla pipeline veya servis aynı branch/commit'i S3-compatible endpoint üzerinden doğrudan okuyabilir; her consumer için ayrı dataset kopyası gerekmez.
Erişim ve yönetişim	+ Mevcut Git ve object-storage kimlik modeline dayanır; ek servis erişimi yoktur.	- Consumer'lara lakeFS endpoint erişimi, credential ve uygun repository/policy izinleri tanımlanmalıdır; network ve yetki yönetimi ek yük oluşturur.

Karar çerçevesi: Lineer veya Git-merkezli geliştirmede DVC daha düşük maliyetli başlangıçtır. Zero-copy data branch'lerinin çok sayıda uzak consumer tarafından ortak kullanılması temel ihtiyaç haline geldiğinde lakeFS'in ek servis maliyeti anlamlı hale gelir.